

SINGLE SOURCE OF TRUTH

SOURCE-A

AI Operating System

Architecture Master Document

نسخه 1.1 – نهایی، قفل شده و توافق شده

Consolidated Edition · Core v1.0 + Post-Review Addendum v1.1

8.7 /10

SSOT Review Score

LOCKED

Production-Ready Thinking

Noetfield Systems Inc. | TrustField Technologies Inc.

Vancouver, BC — 2026

این سند، نسخه‌ی یکپارچه‌ی Single Source of Truth پروژه‌ی Source-A است. بخش‌های ۱ تا ۸ هسته‌ی توافق‌شده‌ی نسخه‌ی ۱.۰ و بخش‌های ۹ تا ۱۴ به‌روزرسانی پس از بازبینی (Addendum v1.1) را در بر می‌گیرند.

No architectural decision may be made without reference to this document.

Core Kernel – v1.0 (Locked)

فلسفه معماری – The 4 Pillars

1

توافق نهایی بر این اصل استوار است: تخصصی‌سازی در لایه‌ی پیکربندی (Configuration) اتفاق می‌افتد، نه در لایه‌ی زیرساخت (Infrastructure).

Pillar	Name	Responsibility
P1	Product	لایه‌ی بیزینس و فرانت‌اند – 777، WitnessBC، TrustField، Noetfield، Forge. فقط پوسته و کانفیگ هستند.
P2	Governance	منطق کنترل کیفیت، تأیید هویت و مدیریت SLA. قوانین بازی را تعریف می‌کند.
P3	Runtime	ورکرهای Stateless که فاقد حافظه هستند. بازوی اجرایی کور که از دیتا دستور می‌گیرد.
P4	Factory	کل اکوسیستم متصل به هم که توسط Data Graph مدیریت می‌شود.

قاعده‌ی طلایی

Workers فقط «muscle» هستند، نه «brain». مغز سیستم، Data Graph در Supabase است.

معماری ۸ لایه‌ای – Source-A Kernel

2

توافق کامل بر این ۸ لایه نهایی شد:

Primary Role	Technology	Name	Layer
داشبورد یکپارچه بر اساس project_id	Next.js + Cloudflare Pages	UI / Interface	L1
امنیت چندمستأجره – Row Level Security	Supabase Auth	Auth & RLS	L2
مدیریت Runs, Tasks, Decisions, Evidence	Supabase Postgres	State Engine	L3
زمان‌بندی، Retry Mechanism، Throttling	Cloudflare Queues	Queue Engine	L4
۴ ورکر: Intake, Planner, Executor, Memory	Stateless CF Workers	Runtime	L5
نگاشت قابلیت‌ها به مدل‌ها (نه نام مدل)	Dynamic Tool Registry	Capability Router	L6
رندر ویدیو – هزینه‌ی صفر تا فراخوانی FFmpeg	Modal / RunPod	Heavy Compute	L7
توکن، هزینه، Audit Trail, Execution Graph	Telemetry + Cost Engine	Observability	L8

۲.۱ SLA-Based Model Router (نه مدل-based)

روتر بر اساس Capability تصمیم می‌گیرد، نه نام مدل. این فیوچر پروفترین طراحی است:

Usage %	Default Provider	SLA Level	Capability
~70%	Gemini Flash	Draft	research / drafting
~20%	Gemini Pro / DeepSeek	Normal	reasoning / analysis
~8%	Claude / OpenAI	Premium	writing / verification
~2%	Claude (Anthropic)	Critical	governance / audit

3 سلسله‌مراتب داده – The Data Hierarchy

این سلسله‌مراتب، ستون فقرات Source-A است. هر سطح، فرزندان خود را در بر می‌گیرد:

```

Tenant (مشتری / سازمان)
└─ Project (برند: Noetfield | Forge | TrustField | WitnessBC | 777)
    └─ Factory (خط تولید تخصصی: Board Review | Risk Scan | Ad Generator)
        └─ Blueprint (قرارداد اجرا: inputs + tools + guards + SLA + workflow)
            └─ Run (نمونه زنده از اجرای Blueprint)
                └─ Task (گام اتمیک محاسباتی)
                    └─ Artifact (فایل: ویدیو | صدا | متن)
                    └─ Evidence (شواهد: دلایل میانی | اثبات)
                    └─ Decision (حکم: approved | rejected | escalated)

```

۳.۱ Blueprint – قرارداد، نه اسکریپت

Blueprint یک قرارداد جامع است. ساختار آن:

```

{
  "blueprint_name": "Forge_Ad_Generator",
  "inputs_schema": { /* فیلدهای ورودی ضروری */ },
  "tools_config": { /* ابزارهای مجاز */ },
  "guards_config": { /* خطوط قرمز و فیلترها */ },
  "outputs_schema": { /* فرمت و نوع خروجی */ },
  "sla_default": "normal",
  "workflow_graph": [
    { "step": 1, "name": "Script_Writing", "capability": "writing", "sla": "normal" },
    { "step": 2, "name": "Voice_Generation", "capability": "tts", "sla": "draft" },
    { "step": 3, "name": "Video_Stitching", "capability": "video_render", "sla": "premium" }
  ]
}

```

قانون اصلی: افزودن Factory جدید = فقط تعریف Blueprint جدید. بدون کد جدید، بدون Worker جدید.

```
-- پروژه‌ها (برندهای هگانه)
CREATE TABLE projects (
  id          UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
  name        VARCHAR(255) NOT NULL,
  slug        VARCHAR(100) UNIQUE NOT NULL,
  metadata    JSONB DEFAULT '{}':::jsonb,
  created_at  TIMESTAMPTZ DEFAULT NOW()
);

-- کارخانه‌ها (خطوط تولید تخصصی)
CREATE TABLE factories (
  id          UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
  project_id  UUID NOT NULL REFERENCES projects(id) ON DELETE CASCADE,
  name        VARCHAR(255) NOT NULL,
  slug        VARCHAR(100) NOT NULL,
  is_active   BOOLEAN DEFAULT TRUE,
  created_at  TIMESTAMPTZ DEFAULT NOW(),
  UNIQUE(project_id, slug)
);
```

```
-- بلوپرینته‌ها (قرارداد جامع خط تولید)
CREATE TABLE blueprints (
  id          UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
  factory_id  UUID NOT NULL REFERENCES factories(id) ON DELETE CASCADE,
  version     INT NOT NULL DEFAULT 1,
  inputs_schema JSONB NOT NULL DEFAULT '{}':jsonb,
  tools_config JSONB NOT NULL DEFAULT '{}':jsonb,
  guards_config JSONB NOT NULL DEFAULT '{}':jsonb,
  outputs_schema JSONB NOT NULL DEFAULT '{}':jsonb,
  workflow_graph JSONB NOT NULL DEFAULT '{}':jsonb,
  is_latest   BOOLEAN DEFAULT TRUE,
  created_at  TIMESTAMPTZ DEFAULT NOW()
);

-- اجراها (نمونه‌های زنده)
CREATE TABLE runs (
  id          UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
  blueprint_id UUID NOT NULL REFERENCES blueprints(id),
  status      VARCHAR(50) NOT NULL DEFAULT 'pending',
              -- pending | running | completed | failed | paused
  current_node_id VARCHAR(100),
  initiated_by  UUID,
  created_at   TIMESTAMPTZ DEFAULT NOW(),
  updated_at   TIMESTAMPTZ DEFAULT NOW()
);

-- تسک‌های اتمیک
CREATE TABLE tasks (
  id          UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
  run_id      UUID NOT NULL REFERENCES runs(id) ON DELETE CASCADE,
  node_id     VARCHAR(100) NOT NULL,
  capability_required VARCHAR(150) NOT NULL,
              -- reasoning | tts | video_render | research | governance
  status      VARCHAR(50) NOT NULL DEFAULT 'pending',
  input_data  JSONB DEFAULT '{}':jsonb,
  output_data JSONB DEFAULT '{}':jsonb,
  retry_count INT DEFAULT 0,
  started_at  TIMESTAMPTZ,
  completed_at TIMESTAMPTZ,
  created_at  TIMESTAMPTZ DEFAULT NOW()
);
```

```
-- آرتیفکت‌ها: فایل‌ها و دیتای ساختاری (ویدیو، صدا، متن)
CREATE TABLE artifacts (
  id          UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
  run_id      UUID NOT NULL REFERENCES runs(id) ON DELETE CASCADE,
  task_id     UUID REFERENCES tasks(id) ON DELETE SET NULL,
  artifact_type VARCHAR(50) NOT NULL, -- video_url | audio_url | raw_text | document
  storage_path TEXT,                  -- آدرس در Cloudflare R2
  payload     JSONB DEFAULT '{}':::jsonb,
  created_at  TIMESTAMPTZ DEFAULT NOW()
);

-- TrustField (حیاتی برای شواهد: لاجیک و دیتای میانی)
CREATE TABLE evidence (
  id          UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
  run_id      UUID NOT NULL REFERENCES runs(id) ON DELETE CASCADE,
  task_id     UUID REFERENCES tasks(id) ON DELETE SET NULL,
  source      VARCHAR(100) NOT NULL,
  payload     JSONB NOT NULL,
  confidence_score NUMERIC(4,3), -- 0.000 1.000 L
  created_at  TIMESTAMPTZ DEFAULT NOW()
);

-- Noetfield (حیاتی برای تصمیمات: خروجی‌های حاکمیتی)
CREATE TABLE decisions (
  id          UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
  run_id      UUID NOT NULL REFERENCES runs(id) ON DELETE CASCADE,
  task_id     UUID REFERENCES tasks(id) ON DELETE SET NULL,
  decision_type VARCHAR(100) NOT NULL,
  verdict     VARCHAR(50) NOT NULL, -- approved | rejected | escalated
  rationale   TEXT NOT NULL,
  created_at  TIMESTAMPTZ DEFAULT NOW()
);
```



```

-- ماشین وضعیت برای جلوگیری از Chaos
CREATE TABLE state_transitions (
  id          UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
  run_id      UUID NOT NULL REFERENCES runs(id) ON DELETE CASCADE,
  from_state  VARCHAR(50) NOT NULL,
  to_state    VARCHAR(50) NOT NULL,
  transition_trigger VARCHAR(255) NOT NULL,
  created_at  TIMESTAMPTZ DEFAULT NOW()
);

CREATE TABLE run_logs (
  id          BIGSERIAL PRIMARY KEY,
  run_id      UUID NOT NULL REFERENCES runs(id) ON DELETE CASCADE,
  level       VARCHAR(10) NOT NULL, -- INFO | WARN | ERROR | CRITICAL
  message     TEXT NOT NULL,
  context     JSONB DEFAULT '{}'::jsonb,
  created_at  TIMESTAMPTZ DEFAULT NOW()
);

CREATE TABLE task_logs (
  id          BIGSERIAL PRIMARY KEY,
  task_id     UUID NOT NULL REFERENCES tasks(id) ON DELETE CASCADE,
  level       VARCHAR(10) NOT NULL,
  message     TEXT NOT NULL,
  context     JSONB DEFAULT '{}'::jsonb,
  created_at  TIMESTAMPTZ DEFAULT NOW()
);

```

```

CREATE TABLE model_calls (
  id          UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
  task_id     UUID NOT NULL REFERENCES tasks(id) ON DELETE CASCADE,
  provider    VARCHAR(100) NOT NULL,    -- Google | DeepSeek | Anthropic
  model_name  VARCHAR(100) NOT NULL,
  capability_used VARCHAR(150) NOT NULL,
  latency_ms  INT NOT NULL,
  created_at  TIMESTAMPTZ DEFAULT NOW()
);

CREATE TABLE token_usage (
  id          UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
  model_call_id UUID NOT NULL REFERENCES model_calls(id) ON DELETE CASCADE,
  prompt_tokens INT NOT NULL DEFAULT 0,
  completion_tokens INT NOT NULL DEFAULT 0,
  estimated_cost_usd NUMERIC(10,7) NOT NULL DEFAULT 0.0,
  created_at  TIMESTAMPTZ DEFAULT NOW()
);

CREATE TABLE execution_traces (
  id          UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
  run_id      UUID NOT NULL REFERENCES runs(id) ON DELETE CASCADE,
  trace_id    VARCHAR(128) NOT NULL,
  span_id     VARCHAR(64) NOT NULL,
  parent_span_id VARCHAR(64),
  operation_name VARCHAR(255) NOT NULL,
  duration_ms  INT NOT NULL,
  created_at  TIMESTAMPTZ DEFAULT NOW()
);
    
```

```

-- Identity Layer
CREATE INDEX idx_factories_project ON factories(project_id);
CREATE INDEX idx_blueprints_factory ON blueprints(factory_id);
-- Execution Layer
CREATE INDEX idx_runs_blueprint ON runs(blueprint_id);
CREATE INDEX idx_runs_status ON runs(status);
CREATE INDEX idx_tasks_run ON tasks(run_id);
CREATE INDEX idx_tasks_status ON tasks(status);
-- Output Layer
CREATE INDEX idx_artifacts_run ON artifacts(run_id);
CREATE INDEX idx_evidence_run ON evidence(run_id);
CREATE INDEX idx_decisions_run ON decisions(run_id);
-- Control + Observability
CREATE INDEX idx_state_transitions_run ON state_transitions(run_id);
CREATE INDEX idx_run_logs_run_level ON run_logs(run_id, level);
CREATE INDEX idx_model_calls_task ON model_calls(task_id);
CREATE INDEX idx_token_usage_call ON token_usage(model_call_id);
CREATE INDEX idx_execution_traces_run ON execution_traces(run_id);
    
```

به جای ۵۰ Worker مجزا، کل کارخانه بر ۴ Stateless Worker می‌چرخد. تخصصی‌سازی در JSON Blueprint است:

W2 Planner Worker

با توجه به Blueprint پروژه، کار را به تسک‌های اتمیک خرد می‌کند و گراف وابستگی می‌سازد.

- خواندن workflow_graph از Blueprint
- ایجاد Task برای هر node در گراف
- تعیین capability_required برای هر Task
- مدیریت وابستگی‌های بین Task‌ها (DAG)

W1 Intake Worker

ورودی (پرامپت، فایل، webhook) را می‌گیرد و یک Run در دیتابیس ثبت می‌کند. دروازه‌ی ورودی سیستم.

- دریافت trigger از API یا Webhook
- اعتبارسنجی ورودی بر اساس inputs_schema بلوپرینت
- ثبت Run با status=pending در دیتابیس
- ارسال run_id به Queue

W4 Memory Worker

خروجی‌ها را برچسب‌گذاری کرده و برای استفاده‌های بعدی Agent ذخیره می‌کند.

- Tag کردن Artifacts با metadata مناسب
- آپلود فایل‌ها به Cloudflare R2
- ذخیره‌ی وکتورهای حافظه در pgvector
- اعلام تکمیل Run از طریق Realtime به فرانت‌اند

W3 Executor Worker

موتور اصلی. capability درخواستی را می‌خواند، Router را می‌پرسد و ابزار مناسب را اجرا می‌کند.

- خواندن capability_required از Task
- ارسال به Capability Router برای انتخاب Provider
- اجرای API call به مدل یا سرویس مربوطه
- ذخیره خروجی در artifacts / evidence / decisions
- فراخوانی Modal برای heavy compute

Goal	Tables / Components	Focus	Phase
Blueprint زنده — اجرای اولین Data Graph	projects, factories, blueprints, runs, tasks, artifacts	Core Kernel	Phase 1
اولین Run کامل از ورودی تا Artifact	Queue + 4 Workers + evidence + decisions	Execution	Phase 2
Multi-model routing بدون hardcode	Capability Router + Tool Registry + SLA	Intelligence	Phase 3
فاکتور دقیق برای مشتری + کنترل هزینه	model_calls, token_usage, execution_traces	Observability	Phase 4

- ✓ **Phase 1:** می‌توانی یک Blueprint تعریف کنی و یک Run ثبت شود.
- ✓ **Phase 2:** ورودی وارد می‌شود، Tasks اجرا می‌شوند، Artifact تولید می‌شود.
- ✓ **Phase 3:** همان Blueprint با Gemini Flash شروع می‌شود و Claude برای گام حساس کنترل می‌گیرد.
- ✓ **Phase 4:** می‌دانی هر Run دقیقاً چقدر هزینه داشت.

7 مقایسه‌ی اقتصادی — Source-A vs Traditional

شاخص	معماری سنتی	Source-A OS
هزینه‌ی ثابت ماهانه	(سرورهای همیشه روشن) \$500–\$1,500	(فقط مصرف واقعی) \$55–\$190
افزودن محصول جدید	سرور + اپ + دیتابیس جدید	یک ردیف در جدول projects
مدیریت ۵۰ نود	(deploy کابوس) مجزا Worker ۵۰	JSON بلوپرینت ۱ Executor + N
مقیاس‌پذیری	کاهش با بار بالا	تا هزاران درخواست Auto-scale
افزودن مدل جدید	تغییر کد در همه‌ی Workerها	یک ردیف در Tool Registry
Audit Trail	ندارد یا دستی	خودکار — evidence + decisions

8 توافقات نهایی قفل‌شده

این بخش صرفاً برای مرجع و جلوگیری از انحراف در جلسات آینده است:

❌ رد شد / حذف شد

- N) ...ElevenLabsWorker، ClaudeWorker، GeminiWorker
Worker برای N ابزار) – رد شد
- Factory = Data Graph – رد شد. «Factory = Code Logic»
- Router مدل محور (if premium: use_claude) – رد شد. باید
Capability-based باشد
- Observability به عنوان Phase 4 اختیاری – رد شد. از روز اول
اجباری است

✅ توافق شد

- Source-A یک **AI Operating System** است، نه یک automation tool
- معماری ۸ لایه نهایی و قفل شد (UI → Auth → State → Queue → Runtime → Router → Compute → Observability)
- سلسله مراتب داده: Tenant → Project → Factory → Blueprint → Run → Task → [Artifact | Evidence | Decision]
- ۴ Worker اصلی کافی است: Intake، Planner، Executor، Memory
- Router بر اساس Capability انتخاب می‌کند، نه نام مدل (future-proof)
- Observability از روز اول – نه بعداً
- Schema v1 نهایی و آماده‌ی deploy به Supabase

🕒 موکول به بعد

- expansion – RWA Tokenization Layer فقط بعد از live شدن core
- Phase 3 – Heavy Compute (Modal / RunPod) به بعد
- Phase 2 – Multi-tenant RLS کامل

Post-Review Addendum — v1.1

Incorporating Critic 1 + Critic 2 Final Decisions

ارزیابی خارجی — Critic Verdicts

9

پس از مرور SSOT v1.0 توسط دو منتقد مستقل، جمع‌بندی زیر به دست آمد:

Reviewer	Score	Verdict
Critic 1 (ASF)	CONFIRMED	معماری به نقطه‌ی build-ready kernel رسیده. اولویت: Executor → Router → DB split
Critic 2	8.7 / 10	فوری Phase 1 شروع. Production-ready thinking with good discipline.

نقاط قوت تأییدشده

- ✓ تفکیک Product / Governance / Runtime / Factory — عالی
- ✓ Blueprint-driven Development (Data Graph) — رویکرد درست برای مقیاس‌پذیری بلندمدت
- ✓ Stateless Worker به جای N Worker تخصصی — تصمیم هوشمند
- ✓ Capability-based Router (نه مدل‌محور) — Future-proof و elegant
- ✓ Observability از روز اول — حیاتی برای تجاری‌سازی

ریسک‌های شناسایی‌شده

- ▲ Over-engineered برای مرحله‌ی اولیه — نیاز به اولویت‌بندی دقیق فازها
- ▲ Executor Worker هنوز Hardening نشده — اولین گلوگاه واقعی سیستم
- ▲ وابستگی به Supabase در مقیاس بالا — قابل مدیریت در Phase 3

اولویت‌بندی ساخت — Revised Build Priority (Locked)

10

توافق Critic 1 و Critic 2: مشکل فعلی infrastructure نیست. مشکل، Runtime Decision Engine است.

Unlocks	Why First	Component	Priority
Router بدون Executor بی‌معناست	همه چیز از اینجا عبور می‌کند. شکست = کل سیستم down	Executor Worker Hardening	#1 – NOW
Cost control + model-agnostic	مغز اقتصادی سیستم. بدون این، هزینه explode می‌شود	Capability Router	#2 – NOW
فاکتور دقیق به مشتری	Full audit trail + cost tracking per Run	State + Observability hardening	#3 – Next
Branching + autoscale	فقط وقتی bottleneck واقعاً به DB رسید	Neon DB Migration	#4 – Later

⚠️ اصلاح مهم 1 Critic به 2 Critic

Neon انتخاب خوبی است اما migration فوری نیست. Bottleneck فعلی، execution logic است، نه Neon. database فقط وقتی اجرا می‌شود که فشار scale واقعی روی DB رسید.

– Production Hardening Spec

Executor Worker

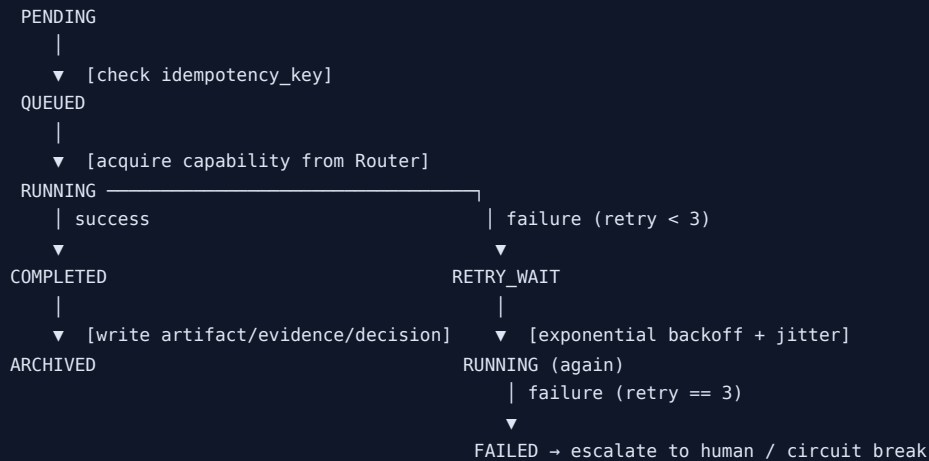
11

اولین پیاده‌سازی واقعی هسته‌ی Source-A. هر Task از این Worker عبور می‌کند.

۱۱.۱ الزامات پایداری (Must-Have)

Purpose	Implementation	Mechanism
جلوگیری از double-execution در retry	چک قبل از هر اجرا – Task بر روی هر UUID	Idempotency Keys
جلوگیری از cascade failure	بعد از N شکست → block provider + fallback	Circuit Breaker
مقاومت در برابر transient errors	حداکثر ۳ بار – jitter با Exponential backoff	Retry + Backoff
جلوگیری از hanging workers	Per-capability timeout – TTS: 10s, Render: 300s	Timeout Budget
شکست یک tool = شکست یک Task، نه کل Run	هر API call در try/catch مستقل	Tool Isolation
Audit trail کامل + debug	ذخیره‌ی input + output کامل در Evidence	Snapshot Before/After
قابلیت جستجو و مانیتورینگ	JSON log با run_id, task_id, level, context	Structured Logging

چرخه‌ی حیات هر Task در Executor:



Executor Worker – Core Logic (Pseudocode) ۱۱.۳

```

async function executeTask(task) {
  // 1. Idempotency guard
  if (await isAlreadyCompleted(task.id)) return;

  // 2. Acquire model from Capability Router
  const provider = await capabilityRouter.resolve({
    capability: task.capability_required,
    sla: task.sla_level,
  });

  // 3. Snapshot input
  await saveEvidence({ task_id: task.id, source: "input", payload: task.input_data });

  // 4. Execute with timeout + circuit breaker
  let result;
  try {
    result = await withTimeout(
      provider.execute(task.input_data),
      TIMEOUT_BUDGETS[task.capability_required]
    );
  } catch (err) {
    await handleFailure(task, err); // retry / circuit break / escalate
    return;
  }

  // 5. Route output to correct table
  if (task.output_type === "artifact") await saveArtifact(task.run_id, task.id, result);
  if (task.output_type === "evidence") await saveEvidence({ task_id: task.id, source: "output", payload: result });
  if (task.output_type === "decision") await saveDecision(task.run_id, task.id, result);

  // 6. Update state + log
  await updateTaskStatus(task.id, "completed");
  await logTask(task.id, "INFO", "Task completed", { provider: provider.name, latency_ms });
}

```


روتر بر اساس Capability تصمیم می‌گیرد + real-time cost feedback. مدل‌ها عوض می‌شوند، Router نه.

۱۲.۱ مدل ترکیبی توصیه‌شده — پس از بررسی Critic 2

Cost Tier	Volume	Default Model	SLA	Capability
♥ Near-zero	~40%	Gemini 3.1 Flash-Lite	draft	intake / validation
♥ Near-zero	~35%	DeepSeek V4 Flash	draft	research / drafting
● Low	~15%	Gemini Pro / DeepSeek	normal	reasoning / analysis
● Medium	~8%	Claude Sonnet 4.6	premium	writing / verification
● Premium	~2%	Claude Sonnet 4.6	critical	governance / audit

نتیجه: ۷۵٪ از حجم کار با near-zero cost انجام می‌شود. Claude فقط ۱۰٪ نهایی را لمس می‌کند.

Capability Router — Core Logic ۱۲.۲

```
// Tool Registry نه در دیتابیس ذخیره است، نه hardcoded
const CAPABILITY_MAP = {
  "intake": { providers: ["gemini-flash-lite"], fallback: "deepseek-flash" },
  "research": { providers: ["deepseek-v4-flash"], fallback: "gemini-pro" },
  "reasoning": { providers: ["gemini-pro", "deepseek-r2"], fallback: "claude-sonnet-4-6" },
  "writing": { providers: ["claude-sonnet-4-6"], fallback: "gemini-pro" },
  "governance": { providers: ["claude-sonnet-4-6"], fallback: null /* escalate */ },
  "tts": { providers: ["elevenlabs"], fallback: "openai-tts" },
  "video_render": { providers: ["modal_ffmpeg"], fallback: null /* queue */ },
};

async function resolve({ capability, sla }) {
  const config = await db.getCapabilityConfig(capability); // از Tool Registry
  const provider = selectProvider(config, sla);
  if (circuitBreaker.isOpen(provider)) {
    return config.fallback ?? escalate(capability);
  }
  return provider;
}
```

اضافه‌شده از Critic 2: real-time cost + latency feedback برای تنظیم داینامیک SLA levels.

تصمیم توافق‌شده: Neon migration فوری نیست. فقط وقتی که bottleneck واقعاً به DB رسید اجرا می‌شود.

Neon (Phase 3+)	Supabase (Now)	Layer
—	✓ Supabase Auth — بماند	Auth & RLS
—	✓ Supabase Realtime — بماند	Realtime / Webhooks
Phase 3: migrate Execution Graph	✓ Supabase Postgres — Phase 1 & 2	State Engine
رسید scale وقتی فشار Neon	✓ Supabase — فعلاً کافی است	Execution Graph (Runs/Tasks)
pgvector در Neon (اگر branch نیاز شد)	✓ pgvector در Supabase	Memory / Vectors
✓ Neon Branching برای تست ایمن	بدون Branching	Blueprint Testing

چرا Neon؟ (برای وقتی که Phase 3 رسید)

- ✓ Database Branching — تست Blueprint جدید بدون ریسک روی production
- ✓ True Serverless + Autoscale to Zero — مناسب بارهای نوسانی ایجنتی
- ✓ Native pgvector — حافظه و جستجوی semantic یکجا
- ✓ Better connection pooling برای صدها worker همزمان

نقشه‌ی راه نهایی به‌روزشده v1.1 —

14

Success Metric	Deliverables	Timeline	Phase
اولین Run کامل: ورودی → Artifact	Core Schema + Executor hardened + Basic Router	2–3 weeks	Phase 1
فاکتور دقیق هر Run در دسترس	Full 4 Workers + Observability + Cost Tracking	2–3 weeks	Phase 2
تغییر مدل بدون تغییر کد	Advanced Router + Tool Registry + SLA dynamic	3–4 weeks	Phase 3
فعال Branching + autoscale	Neon Migration (if scale pressure) + Heavy Compute	On-demand	Phase 4

Executor Worker Hardening شروع با

چون همه چیز از اینجا عبور می‌کند.

SOURCE-A KERNEL v1.1 – LOCKED

این سند، SSOT رسمی پروژه‌ی Source-A است. هیچ تصمیم معماری بدون ارجاع به این سند نباید گرفته شود.

.Noetfield Systems Inc. | TrustField Technologies Inc

Vancouver, BC – 2026